Original_Torch_Implementation.py

```python
import torch
import torch.nn as nn

class Model(nn.Module):
"""
Simple model that performs L1 normalization.
"""
def __init__(self):
"""
Initializes the L1 normalization layer.
"""
super(Model, self).__init__()

def forward(self, x: torch.Tensor) -> torch.Tensor:
"""
Applies L1 normalization to the input tensor.

Args:
x (torch.Tensor): Input tensor of shape (..., dim, ...).

Returns:
torch.Tensor: Output tensor with L1 normalization applied, same shape as input.
"""
return x / torch.mean(torch.abs(x), dim=1, keepdim=True)

batch_size = 32768
# choose dim so total <2^31
dim = 65535

def get_inputs():
x = torch.rand(batch_size, dim)
return [x]

def get_init_inputs():
return []
```

1st Turn Trajectory (1)

Thinking, analysis, and plan

- What the original PyTorch code does
  - It computes mean(abs(x), dim=1, keepdim=True) for a 2D tensor x of shape [B, D].
  - ...

- Where the time goes in the PyTorch version
  - ...
  - So you have multiple passes over memory and intermediate tensors, which for large [B, D] are memory-bandwidth bound.

- What we can do better with Triton
  - Fuse abs, reduction, and scaling into a single kernel that:
    - Reads each element of a row once to accumulate the L1 norm (sum of abs).
    - Computes the inverse mean (1/mean) from that sum.
    - Makes a second pass over the row to write out x / mean.
  - This removes materialization of abs(x) and mean, and reduces kernel launch overhead.
  - Memory access is simple and coalesced: process one row per program, iterate over columns in blocks.

- Kernel structure
  - Grid: one program per row (grid = (B,)).
  - Loop over columns in chunks of BLOCK_N (e.g., 128/256/512/1024).
  - ...

- Tuning
  - BLOCK_N: choose based on D (next power-of-two up to a cap), e.g., 128–1024.
  - num_warps: 4 or 8 is typical; 4 is fine for many cases.
  - This is a simple, robust choice; autotuning is possible but overkill here.

- Edge cases and correctness
  - ...

Code (Triton-optimized L1 row normalization)

```python
    ...
    @triton.jit
    def _l1_norm_rows_kernel(
        x_ptr,           # *const T
        y_ptr,           # *T
        B: tl.constexpr,    # number of rows
        D: tl.constexpr,    # number of cols
        stride_b: tl.constexpr,  # row stride in elements
        stride_d: tl.constexpr,  # col stride in elements
        BLOCK_N: tl.constexpr
    ):
        # program id = row index
        row = tl.program_id(0)
        # guard: if grid is larger than B (shouldn't happen with correct grid), early return
        if row >= B:
            return

        # Accumulate sum of abs over this row in float32
        sum_val = tl.zeros((), dtype=tl.float32)

        # First pass: compute L1 norm = sum |x|
        start = 0
```

1st Turn Trajectory (2)

```python
        while start < D:
            offs = start + tl.arange(0, BLOCK_N)
            mask = offs < D
            ptrs = x_ptr + row * stride_b + offs * stride_d
            vals = tl.load(ptrs, mask=mask, other=0.0)
            vals_f32 = vals.to(tl.float32)
            block_sum = tl.sum(tl.abs(vals_f32), axis=0)
            sum_val += block_sum
            start += BLOCK_N

        # mean = sum / D ; inv_mean = 1 / mean
        # guard D > 0 (host should ensure); compute in f32
        inv_mean = 1.0 / (sum_val / D)

        # Second pass: y = x * inv_mean
        start = 0
        while start < D:
            offs = start + tl.arange(0, BLOCK_N)
            mask = offs < D
            in_ptrs = x_ptr + row * stride_b + offs * stride_d
            out_ptrs = y_ptr + row * stride_b + offs * stride_d
            vals = tl.load(in_ptrs, mask=mask, other=0.0)
            vals_f32 = vals.to(tl.float32)
            out_vals = vals_f32 * inv_mean
            # cast back to input dtype
            out_cast = out_vals.to(vals.dtype)
            tl.store(out_ptrs, out_cast, mask=mask)
            start += BLOCK_N
    def _choose_block_n(D: int) -> int:
        # next power-of-two up to 1024, minimum 64
        if D <= 64:
            return 64
        p2 = 1 << (int(math.ceil(math.log2(max(1, D)))))
        return min(max(64, p2), 1024)

    class ModelNew(nn.Module):
        def __init__(self):
            super().__init__()

        def forward(self, x: torch.Tensor) -> torch.Tensor:
            # Fallback to PyTorch if not on CUDA or Triton not available
            if (not x.is_cuda) or (not _HAS_TRITON):
                # match original: mean over dim=1, keepdim=True, then divide
                return x / torch.mean(torch.abs(x), dim=1, keepdim=True)

            assert x.dim() == 2, f"Expected 2D tensor, got shape {tuple(x.shape)}"
            B, D = x.shape
            if D == 0:
                raise ValueError("Normalization dimension D must be > 0")
            if not x.is_contiguous():
                x = x.contiguous()

            y = torch.empty_like(x)

            BLOCK_N = _choose_block_n(D)
            grid = (B,)
            _l1_norm_rows_kernel[grid](
                x, y,
                B, D,
                x.stride(0), x.stride(1),
                BLOCK_N=BLOCK_N,
                num_warps=4,  # reasonable default
            )
            return y
```

2nd Turn Trajectory (1)

Deep dive analysis, thinking, and optimization plan

1) What the original PyTorch model does
- ...

2) Performance characteristics of the PyTorch version
- ...

3) First Triton version (your feedback context)
- ...
- This removes intermediate tensors and kernel launches, gives a small speedup (~1.04x on H100 for B=32768, D=65535).

4) Where the time is spent and how to improve
- ...

5) Optimization plan
- Autotune BLOCK_N, num_warps, num_stages for the given D.
  - Provide a small set of configs that cover 128–1024 block sizes, warps 4–8, stages 2–4.
  - Key on D so the best config is cached per shape.
- Minor micro-optimizations:
  - Compute inv_mean as D / sum (avoid division earlier).
  - Keep accumulation in float32; cast on store.
  - Ensure contiguous layout.
  - Use while loop over cols; it's fine and compiles well.
- Keep API and correctness identical.

6) Correctness and edge cases
- ...

7) Expected result
- ...

Improved Triton implementation with autotuning

```python
...


@triton.autotune(
    configs=[
        triton.Config({'BLOCK_N': 128},  num_warps=4, num_stages=2),
        triton.Config({'BLOCK_N': 256},  num_warps=4, num_stages=2),
        triton.Config({'BLOCK_N': 512},  num_warps=8, num_stages=3),
        triton.Config({'BLOCK_N': 1024}, num_warps=8, num_stages=4),
    ],
    key=['D'],  # tune per column size
)
@triton.jit
def _l1_norm_rows_kernel(
    x_ptr,           # *const T
    y_ptr,           # *T
    B,               # number of rows (int)
    D,               # number of cols (int)
    stride_b,        # row stride in elements (int)
    stride_d,        # col stride in elements (int)
    BLOCK_N: tl.constexpr,
):
    # program id = row index
    row = tl.program_id(0)
    if row >= B:
        return
```

2nd Turn Trajectory (2)

```python
    # First pass: compute sum |x| in f32
    sum_val = tl.zeros((), dtype=tl.float32)
    start = 0
    while start < D:
        offs = start + tl.arange(0, BLOCK_N)
        mask = offs < D
        ptrs = x_ptr + row * stride_b + offs * stride_d
        vals = tl.load(ptrs, mask=mask, other=0.0)
        vals_f32 = vals.to(tl.float32)
        block_sum = tl.sum(tl.abs(vals_f32), axis=0)
        sum_val += block_sum
        start += BLOCK_N

    # Compute inv_mean = D / sum ; avoid early division
    inv_mean = (D * 1.0) / sum_val

    # Second pass: y = x * inv_mean
    start = 0
    while start < D:
        offs = start + tl.arange(0, BLOCK_N)
        mask = offs < D
        in_ptrs = x_ptr + row * stride_b + offs * stride_d
        out_ptrs = y_ptr + row * stride_b + offs * stride_d
        vals = tl.load(in_ptrs, mask=mask, other=0.0)
        vals_f32 = vals.to(tl.float32)
        out_vals = vals_f32 * inv_mean
        out_cast = out_vals.to(vals.dtype)
        tl.store(out_ptrs, out_cast, mask=mask)
        start += BLOCK_N


class ModelNew(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        ...

        y = torch.empty_like(x)

        grid = (B,)
        _l1_norm_rows_kernel[grid](
            x, y,
            B, D,
            x.stride(0), x.stride(1),
            # Meta-parameters are selected by autotune; no need to pass BLOCK_N/num_warps here
        )
        return y
```

What changed and why it should be faster
- Autotuning over BLOCK_N, num_warps, and num_stages lets Triton pick the best configuration for the given D on the current GPU.
- Increased warps (up to 8) and pipelining (up to 4 stages) help hide memory latency and improve throughput on H100.
- Slight arithmetic tweak (inv_mean = D / sum) avoids an earlier division.
- Everything else remains fused and memory-efficient: two passes, no intermediate tensors, coalesced access.

This should preserve correctness while extracting a bit more performance compared to the fixed-parameter kernel.

3rd Turn Trajectory

Deep analysis, reasoning, and optimization plan

1) Restating the operation
- ...

2) Performance characteristics and bottlenecks
- ...

3) What we did previously and results
- Fused kernel with two passes, float32 accumulation, coalesced access.
- Added autotuning over BLOCK_N, num_warps, num_stages.
- Achieved ~1.21x speedup on H100 for B=32768, D=65535.

4) Where we can squeeze more performance
- Increase instruction-level parallelism and memory pipeline depth:
  - Larger BLOCK_N (up to 2048/4096) can improve vectorization and reduce loop iterations for very large D.
  - Increase num_warps (up to 16) and num_stages (up to 5) to better hide memory latency on H100.
- Minor arithmetic tweak:
  - Compute inv_mean = D / sum to avoid an earlier division.
- Keep the kernel simple to allow the compiler to generate efficient code (contiguous, unit stride along columns).

5) Design decisions
- ...

6) Correctness and edge cases
- ...

7) Expected outcome
- On H100, expanding the search space (BLOCK_N up to 2048, warps up to 16, stages up to 5) should find a configuration that is equal or faster than the previous one, improving the speedup while preserving correctness.

Final improved code (Triton-optimized, with expanded autotuning)

```python
import torch
import torch.nn as nn

try:
    import triton
    import triton.language as tl
    _HAS_TRITON = True
except Exception:
    _HAS_TRITON = False


@triton.autotune(
    configs=[
        triton.Config({'BLOCK_N': 128},  num_warps=4,  num_stages=2),
        triton.Config({'BLOCK_N': 256},  num_warps=4,  num_stages=2),
        triton.Config({'BLOCK_N': 512},  num_warps=8,  num_stages=3),
        triton.Config({'BLOCK_N': 1024}, num_warps=8,  num_stages=4),
        triton.Config({'BLOCK_N': 2048}, num_warps=16, num_stages=4),
        # If D is very large, 4096 can help reduce loop iters; include it sparingly
        triton.Config({'BLOCK_N': 4096}, num_warps=16, num_stages=5),
    ],
    key=['D'],  # tune per column size
)
@triton.jit
def _l1_norm_rows_kernel(
    ...
```